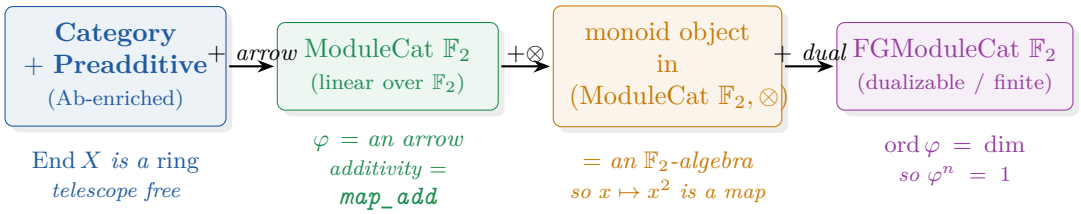
A MacLane-style Build of the Enrichment Ladder

From the bare definition of a category to $\varphi^n = 1$

— `DobbertinLego/MacLaneLadder.lean`

The move. Do not *assume* `[Field F] [Fintype F]`. Start from the definition of a category and add *one axiom per rung.* Each rung absorbs an input into the ambient object, so that additivity, the telescope, and finite order $\varphi^n = 1$ become *theorems about the object*, not hypotheses. Field and finiteness survive only where they are genuinely irreducible — and even `Fintype` and `CharP` are then *derived*, not imposed.

1 The ladder



	added axiom	Mathlib category	buys (a theorem)	Lean name
1	Hom-sets abelian, $\circ$ bilinear	`Category + Preadditive`	telescope $(\varphi-1)\Sigma\varphi^i = \varphi^n-1$	`telescope`
2	linear over $\mathbb{F}_2$	`ModuleCat (ZMod 2)`	φ is an arrow; additivity free	`frob`, `frobArrow`
3	monoidal $\otimes$, monoid object	`MonObj (ModuleCat.of …)`	$x \mapsto x^2$ is a morphism	`monObj`
4	has a dual (finite dim)	`FGModuleCat (ZMod 2)`	$\varphi^n = 1$; Tr from eval/coeval	`fgObj`, `frob_pow_finrank`

2 Rung 1 — Ab: the telescope is the geometric series

The *only* axiom is preadditivity: Hom-sets are abelian groups and composition is bilinear. Then End X is a ring, and the telescope is `mul_geom_sum`. No field, no characteristic, no finiteness.

```
theorem telescope {C : Type*} [Category C] [Preadditive C] {X : C}
    (φ : End X) (n : ℕ) :
    (φ - 1) * ∑ i ∈ range n, φ ^ i = φ ^ n - 1 := mul_geom_sum φ n
```

3 Rung 2 — ModuleCat $\mathbb{F}_2$: Frobenius is an arrow, additivity is free

Work over the base object $\mathbb{F}_2$ = `ZMod 2`. The Frobenius $x \mapsto x^2$ is an element `frob` of the endomorphism ring Module . End $\mathbb{F}_2$ R; its additivity is `map_add` (the freshman's dream, supplied by the module structure). The endomorphism ring *is* the object's categorical End:

$$\mathrm{End}(\texttt{ModuleCat.of}\ \mathbb{F}_2\ R) \xrightarrow[\simeq]{\texttt{endRingEquiv}} \texttt{Module}\,.\,\mathrm{End}\ \mathbb{F}_2\ R$$

$$\texttt{frobArrow} \longmapsto \texttt{frob}$$

so the Rung-1 telescope, run on the arrow, transports verbatim.

```
noncomputable def frob : Module.End 𝔽₂ R where
  toFun x := x ^ 2
  map_add' x y := by simpa using add_pow_char (R := R) (p := 2) x y   -- freshman's dream
  map_smul' r x := by ...                                             -- 𝔽₂-linear

lemma frobArrow_telescope (n : ℕ) :
    (frobArrow - 1) * ∑ i ∈ range n, frobArrow ^ i = frobArrow ^ n - 1 :=
  telescope frobArrow n      -- Rung 1, read inside End (ModuleCat.of 𝔽₂ R)
```

No field yet. `frob` is defined for *any* `[CommRing R] [Algebra (ZMod 2) R] [CharP R 2]`. Additivity is `map_add`; $\mathbb{F}_2$-linearity is $r^2 = r$ in $\mathbb{F}_2$ (`ZMod.pow_card`).

4 Rung 3 — commutative monoid object: why $x \mapsto x^2$ is a map

Make R a *monoid object* of the monoidal category (ModuleCat $\mathbb{F}_2, \otimes$). Via Mathlib's `MonModuleEquivalenceAlgebra`, monoid objects here *are* $\mathbb{F}_2$-algebras — which is precisely what makes squaring a morphism.

$$\text{monoid object of } (\text{ModuleCat } \mathbb{F}_2, \otimes) \xrightarrow[\simeq]{\texttt{MonModuleEquivalenceAlgebra}} \mathbb{F}_2\text{-algebra } R$$

```
noncomputable def monObj : MonObj (ModuleCat.of 𝔽₂ R) :=
  ModuleCat.MonModuleEquivalenceAlgebra.inverseObj (AlgCat.of 𝔽₂ R)
```

5 Rung 4 — dualizable / finite: `Fintype` and $\varphi^n = 1$ derived

A finite-dimensional object of `FGModuleCat` $\mathbb{F}_2$ has a right dual. From `[Module.Finite (ZMod 2) F]` alone, `Finite F`, `Fintype F` and `CharP F 2` are *derived* — nothing about counting is assumed:

$$\texttt{Module.Finite}\ \mathbb{F}_2\ F \xrightarrow{\texttt{finite\_of\_finite}} \texttt{Finite F} \xrightarrow{\texttt{ofFinite}} \texttt{Fintype F}$$

$$\texttt{Algebra}\ \mathbb{F}_2\ F \xrightarrow{\texttt{charP\_of\_inj\_algebraMap}} \texttt{CharP F 2}$$

Then the annihilating polynomial $X^n - 1$ is read off the dimension through a *monoid map* carrying the algebra Frobenius to `frob`:

$$\begin{array}{ccc} (F \to_a [\mathbb{F}_2]F) & \xrightarrow[\to^*]{\texttt{algEndToLin}} & \text{Module.End } \mathbb{F}_2\ F \\ \Big\downarrow{\scriptstyle \text{ord=dim}} & & \Big\downarrow{\scriptstyle (-)^n} \\ \texttt{frobeniusAlgHom} & \longmapsto & \texttt{frob},\quad \texttt{frob}^n = 1 \end{array}$$

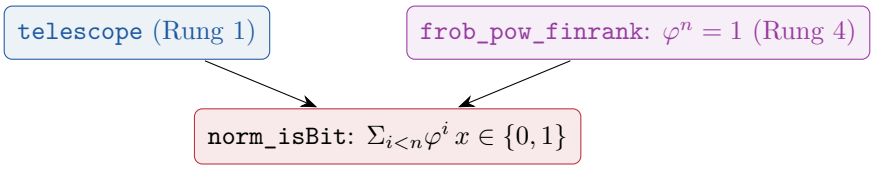
```
instance : Finite F := Module.finite_of_finite 𝔽₂            -- from finite dimension
noncomputable instance : Fintype F := Fintype.ofFinite F      -- Fintype DERIVED
instance : CharP F 2 := charP_of_injective_algebraMap ... 2   -- CharP DERIVED

lemma frob_pow_finrank : (frob (R := F)) ^ (Module.finrank 𝔽₂ F) = 1 := by
  rw [frob_eq_frobenius, ← map_pow, ← frob_orderOf, pow_orderOf_eq_one, map_one]
```

What stays. `Field F` remains at Rung 4: a dualizable object of `FGModuleCat` $\mathbb{F}_2$ is only an algebra, and $\varphi^n = 1$ genuinely fails for non-reduced algebras (e.g. $\mathbb{F}_2[t]/(t^2)$, where $t \mapsto 0$). This is the honest irreducible core — but `Fintype` and `CharP` are no longer assumed.

6 Capstone — the norm element is a bit

Telescope + $\varphi^n = 1 \Rightarrow$ the norm element $\Sigma_{i<n}\varphi^i$ is fixed by φ, i.e. lands in $\{0, 1\}$ — the "trace is a bit", now a consequence of the object's structure.




```
lemma norm_isBit (x : F) :
    (∑ i ∈ range (Module.finrank 𝔽₂ F), frob ^ i) x = 0
      ≠ (∑ i ∈ range (Module.finrank 𝔽₂ F), frob ^ i) x = 1
```

7 Assumed vs. derived

classical hypothesis	here
`[Fintype F]`	derived from `[Module.Finite (ZMod 2) F]`
`[CharP F 2]`	derived from `[Algebra (ZMod 2) F]`
additivity of Frobenius (char 2, by hand)	free: `map_add` of the arrow `frob`
telescope (bespoke induction)	free: `mul_geom_sum` in End X
$x \mapsto x^2$ is a ring map	structure: the monoid object `monObj`
$\varphi^n = 1$ (Fermat, cardinality)	ord = dim of the dualizable object
`[Field F]`	*kept* — irreducible (Rung 4)

Soundness. `DobbertinLego/MacLaneLadder.lean` builds `sorry`-free; `telescope`, `frob_pow_finrank` and `norm_isBit` depend only on `propext`, `Classical.choice`, `Quot.sound`.